

KANDIDATNUMMER(E)/NAVN:

10010

DATO:

22.05.2024

FAGKODE:

IDATG2003

STUDIUM:

Bachelor i ingeniørfag - data

ANT SIDER/BILAG:

27

FAGLÆRER(E) :

Ivar Farup

TITTEL :

Utvikling av Chaos Game

SAMMENDRAG:

I dette prosjektet har jeg utviklet "Chaos Game", en desktop-applikasjon for visualisering av fraktaler. Rapporten beskriver hele utviklingsprosessen, fra problemstilling og formål til de endelige resultatene og konklusjonene. Målet med prosjektet var å lage en applikasjon som kan opprette, visualisere og manipulere fraktaler basert på matematiske transformasjoner.

Den teoretiske delen av rapporten tar for seg viktige prinsipper innen objektorientert programmering, designmønstre og systemarkitektur, som har vært fundamentale i design og implementasjon av systemet. Avslutningsvis drøfter rapporten utfordringene som ble møtt, samtidig de viktigste lærdommene og anbefalingene for fremtidig arbeid.

Denne rapporten gir en helhetlig oversikt over prosjektet og kan leses isolert for å forstå de viktigste resultatene og konklusjonene.

Denne oppgaven er en besvarelse utført av student(er) ved NTNU.

Deklarasjon om KI-hjelpemidler

Har det i utarbeidingen av denne rapporten blitt anvendt KI-baserte hjelpemidler?

☒ nei Nei

☐ Ja

Hvis ja: spesifiser type av verktøy og bruksområde under.

Tekst

☐ **Stavekontroll.** Er deler av teksten kontrollert av:
Grammarly, Ginger, Grammarbot, LanguageTool, ProWritingAid, Sapling, Trinkai.ai eller lignende verktøy?

☐ **Tekstgenerering.** Er deler av teksten generert av:
ChatGPT, GrammarlyGO, Copy.AI, WordAi, WriteSonic, Jasper, Simplified, Rytr eller lignende verktøy?

☐ **Skriveassistanse.** Er en eller flere av ideene eller fremgangsmåtene i oppgaven foreslått av:
ChatGPT, Google Bard, Bing chat, YouChat eller lignende verktøy?

Hvis ja til anvendelse av et tekstverktøy - spesifiser bruken her:

Kode og algoritmer

☒ **Ja** **Programmeringsassistanse.** Er deler av koden/algoritmene som i) fremtrer direkte i rapporten eller ii) har blitt anvendt for produksjon av resultater slik som figurer, tabeller eller tallverdier blitt generert av: *GitHub Copilot, CodeGPT, Google Codey/Studio Bot, Replit Ghostwriter, Amazon CodeWhisperer, GPT Engineer, ChatGPT, Google Bard* eller lignende verktøy?

Hvis ja til anvendelse av et programmeringsverktøy - spesifiser bruken her:

Brukt Chat-GPT som en læringsressurs til å få forslag på mulige forbedringer jeg kunne gjøre. Det har hjulpet meg med å skjønne komplekse konsepter og utvide min kunnskap.

Bilder og figurer

☐ **Bildegenerering.** Er ett eller flere av bildene/figurene i rapporten blitt generert av: *Midjourney, Jasper, WriteSonic, Stability AI, Dall-E* eller lignende verktøy?

Hvis ja til anvendelse av et bildeverktøy - spesifiser bruken her:

Andre KI-verktøy

☐ **Andre KI-verktøy.** har andre typer av verktøy blitt anvendt? Hvis ja spesifiser bruken her:

☐

Jeg er kjent med NTNUs regelverk: *Det er ikke tillatt å generere besvarelse ved hjelp av kunstig intelligens og levere den helt eller delvis som egen besvarelse.* Jeg har derfor redegjort for all anvendelse av kunstig intelligens enten i) direkte i rapporten eller ii) i dette skjemaet

Underskrift/Dato/Sted

INNHold

Innholdsfortegnelse

1	Introduksjon.....	1
1.1	Bakgrunn.....	1
1.2	Kravspesifikasjon	2
1.3	Avgrensninger	3
2	Teori.....	4
3	Metode.....	6
3.1	Utviklingsprosess.....	6
4	Resultat.....	7
4.1	Teknisk Design.....	7
4.2	Implementasjon.....	10
4.3	Testing.....	18
5	Drøfting.....	20
5.1	Drøfting av løsning/design.....	20
5.2	Drøfting av prosess.....	21
6	Konklusjon	22
6.1	Drøfting av bruken av KI-verktøy	22

Figurliste

Figur 1: Use Case diagram av Chaos Game som demonterer funksjonelle og ikke-funksjonelle krav.....	2
Figur 2: Illustrasjon av Systemarkitekturen som har blitt benyttet i systemet.	7
Figur 3: Pakkediagram som illustrerer strukturen mellom pakkene	9
Figur 4: Klassediagram for Brukerinteraksjon	12
Figur 5: Klassediagram for Forretningslogikken (Modellen)	14
Figur 6: Aktivitetsdiagram for Opprettelse av Fraktaler	16
Figur 7: Sekvensdiagram, Generering av Fraktaler.....	17

1 INTRODUKSJON

1.1 Bakgrunn

Applikasjonen «Chaos Game» er et desktop-programvaresystem for å generere fraktaler med ulike mønstre. Denne rapporten beskriver utviklingen av denne applikasjonen, som skaper komplekse, selv-repeterende mønstre gjennom matematiske transformasjoner.

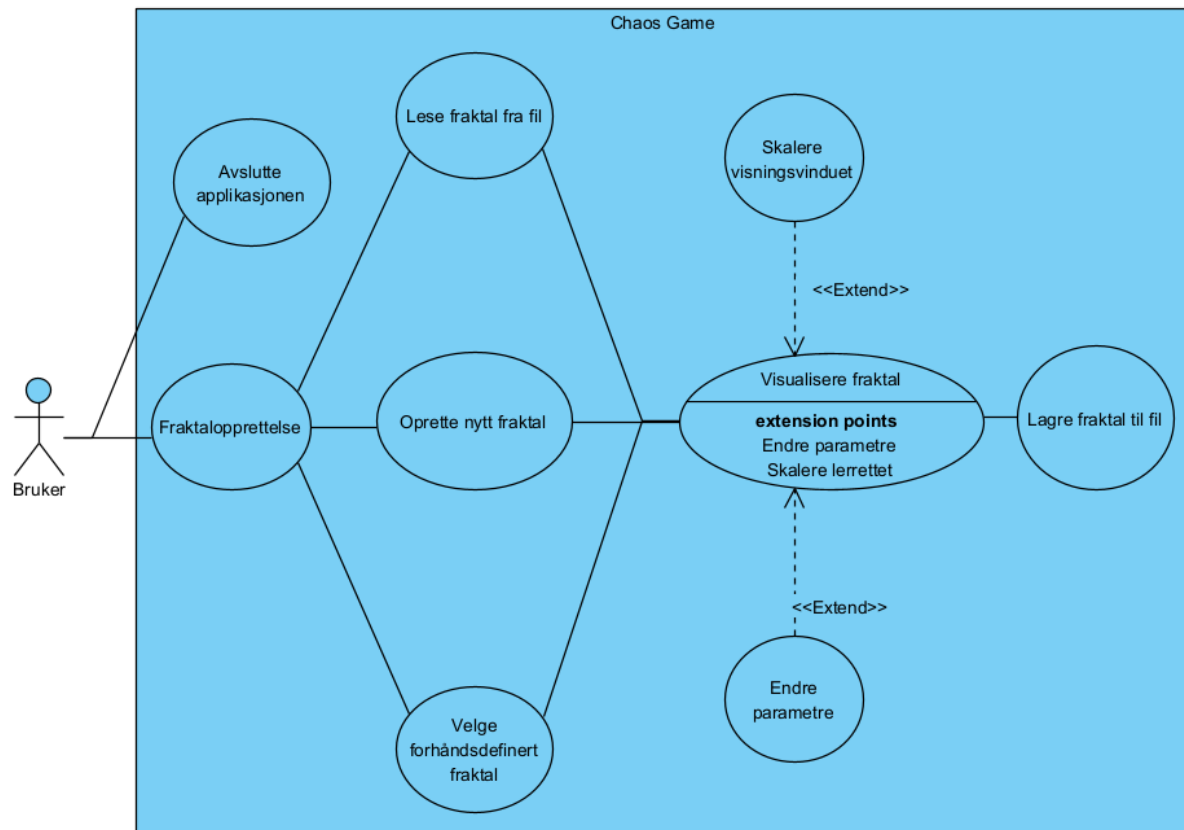
Oppgaven er å utvikle en desktop-applikasjon som følger MVC-arkitekturen, for klar separasjon av ansvar mellom modell, visning og kontroll. Applikasjonen skal la brukere generere og justere parametere for ulike fraktaler, inkludert Julia-mengder og fraktaler basert på affine transformasjoner. Brukere skal også kunne lagre og laste fraktaler fra filer.

Prosjektet gir praktisk erfaring med designprinsipper og designmønstre. Det sikrer også en brukervennlig applikasjon som oppfyller både funksjonelle krav til fraktal generering og -visualisering, samt ikke-funksjonelle krav til robusthet, ytelse og vedlike holdbarhet.

Rapporten dokumenterer utviklingsprosessen, tekniske løsninger og utfordringer, og gir en helhetlig forståelse av programvareutvikling fra start til slutt.

1.2 Kravspesifikasjon

Dette kapittelet beskriver de funksjonelle og ikke-funksjonelle kravene til applikasjonen. Kravene er utarbeidet for å sikre at systemet møter brukernes behov og fungerer effektivt.



Figur 1: Use Case diagram av Chaos Game som demonterer funksjonelle og ikke-funksjonelle krav.

Funksjonelle krav

1. Opprettelse av fraktaler:

- Applikasjonen skal la brukeren opprette nye tomme fraktaler fra menyen, med valg mellom Julia- og affine transformasjoner.
- Brukeren skal kunne velge mellom forhåndsdefinerte fraktaler som Julia, Barnsley og Sierpinski.

2. Visualisering:

- Systemet skal visualisere fraktaler som bilder.

- Brukeren skal kunne endre parametere som antall steg, koordinater, konstanten c for Julia-mengder, samt matrisene A og vektorene b for affine transformasjoner.

3. *Skalering og tilpasning:*

- Brukeren skal kunne skalere og endre størrelsen på visningsvinduet etter behov.

4. *Filhåndtering:*

- Applikasjonen skal inkludere funksjoner for å lese inn og lagre fraktaler med spesifikke parametere.

Ikke-funksjonelle krav

1. *Arkitektur:*

- Applikasjonen skal benytte MVC-arkitekturen for å sikre en tydelig separasjon av bekymringer og bedre organisering av koden.

2. *Robusthet:*

- Systemet skal være robust, håndtere feil uten å krasje, inkludert feil i brukerininput og andre uventede situasjoner.

3. *Brukervennlighet:*

- Brukergrensesnittet skal være enkelt å navigere med tydelige instruksjoner og hjelpetekster der det er nødvendig.

4. *Vedlike holdbarhet:*

- Koden skal være godt strukturert og dokumentert for å gjøre det enkelt å vedlikeholde og utvide applikasjonen i fremtiden.

Disse kravene sikrer at applikasjonen blir funksjonell, brukervennlig, vedlikeholdbar, robust og ytelseseffektiv, noe som er avgjørende for å levere et vellykket prosjekt.

1.3 Avgrensninger

Prosjektet hadde noen spesifikke avgrensninger som påvirket valg av teknologi og arkitektur. Vi måtte bruke MVC-arkitektur (Model-View-Controller), som skiller applikasjonen i tre

hovedkomponenter: modell, visning, og kontroller. Dette ble gjort for å sikre en klar struktur og bedre organisering av koden.

I tillegg var det påkrevd å bruke GitLab for kildekodehåndtering og samarbeid. Dette innebar å bruke GitLab for versjonskontroll, issue tracking, og kontinuerlig integrasjon for å sikre en strukturert og sporbar utviklingsprosess.

2 TEORI

I dette kapitlet vil jeg utforske det teoretiske grunnlaget som støtter arbeidet med designet av systemet. Disse teoriene vil bli referert til i senere kapitler. Vi vil se nærmere på prinsipper og konsepter som er avgjørende for å skape en velstrukturert programvare.

Kobling og Kohesjon

Kobling refererer til sammenkoblingen av klasser, og målet er løs kobling der klassene er uavhengige og kommuniserer via veldefinerte grensesnitt. Kohesjon refererer til hvor godt en enhet samsvarer med en logisk oppgave, og enheter bør ha ansvar for en klart definert oppgave. For å oppnå lav kobling og høy kohesjon, unngå kodeduplisering, kapsle felter og metoder riktig, designe klasser med veldefinerte ansvarsområder og lokalisere endringer til enkeltklasser [1, pp. 255-293], [2], [3], [4].

Arv

Arv i objektorientert programmering tillater en klasse å arve egenskaper og metoder fra en annen klasse, som fremmer gjenbruk av kode og forbedrer organiseringen. Fordelene inkluderer gjenbruk av kode, forbedret organisering, enklere vedlikehold og polymorfisme, hvor objekter av forskjellige klasser kan behandles som objekter av en felles superklasse [1, pp. 343-349]. Arv gjør det mulig å bruke eksisterende kode på nytt, noe som reduserer behovet for å skrive samme kode flere ganger [1, pp. 351-352], [5]. Det etablerer et tydelig hierarki og gjør det lettere å vedlikeholde systemet [1, pp. 351-352], [6].

Abstraksjon, Abstrakte Klasser og Grensesnitt

Abstraksjon forenkler systemer ved å fokusere på essensielle aspekter og skjule irrelevante detaljer. Abstrakte klasser kan inneholde både abstrakte og konkrete metoder, mens grensesnitt definerer et sett med abstrakte metoder som en klasse må implementere. Fordelene med abstrakte klasser og grensesnitt inkluderer tvingende implementering, gjenbruk av kode og fleksibilitet [1, pp. 405-406], [1, pp. 409-412], [1, pp. 417-425].

MVC Arkitektur (Model-View-Controller)

MVC skiller applikasjonen i tre hovedkomponenter:

- Modellen: Håndterer data og forretningslogikk, uavhengig av brukergrensesnittet.
- Visningen: Presenterer data fra modellen til brukeren uten å manipulere dataene.
- Kontrolleren: Håndterer brukerinput, oppdaterer modellen basert på input og bestemmer hvilken visning som skal vises.

Denne separasjonen av bekymringer forbedrer organiseringen av koden og gjør systemet lettere å teste og vedlikeholde [7].

Ved å bruke prinsippene om lav kobling og høy kohesjon, arv, abstraksjon og MVC-arkitektur, har vi sikret en godt strukturert, fleksibel og vedlikeholdbar applikasjon.

3 METODE

I dette kapitlet vil jeg ta for meg utviklingsprosessen og hva som skal til for å kunne reproducere resultatet som har blitt oppnådd.

3.1 *Utviklingsprosess*

Prosjektet startet med to medlemmer, men etter første godkjenningssamtale jobbet jeg alene. Vi valgte en iterativ og inkrementell prosessmodell for tidlig tilbakemelding og justeringer.

Prosessmodell Implementering:

1. Planlegging: Ukentlige møter for å diskutere fremdrift og utfordringer.
2. Design og Implementasjon: Hver iterasjon inkluderte design av funksjoner og implementasjon.
3. Testing og Tilbakemelding: Koden ble testet og tilbakemeldinger brukt for forbedringer.

Arbeid etter Gruppeoppløsning: Jeg fortsatte å følge den iterative modellen:

1. Planlegging: Opprettet en arbeidsplan med milepæler for design, implementasjon, testing, og dokumentasjon.
2. Implementasjon: Jevn integrasjon med commits til et Git-repository.
3. Dokumentasjon: Dokumenterte utviklingsprosessen, utfordringer og løsninger.
4. Testing: Utførte enhetstester, integrasjonstester og manuelle tester for å sikre funksjonalitet og identifisere feil.

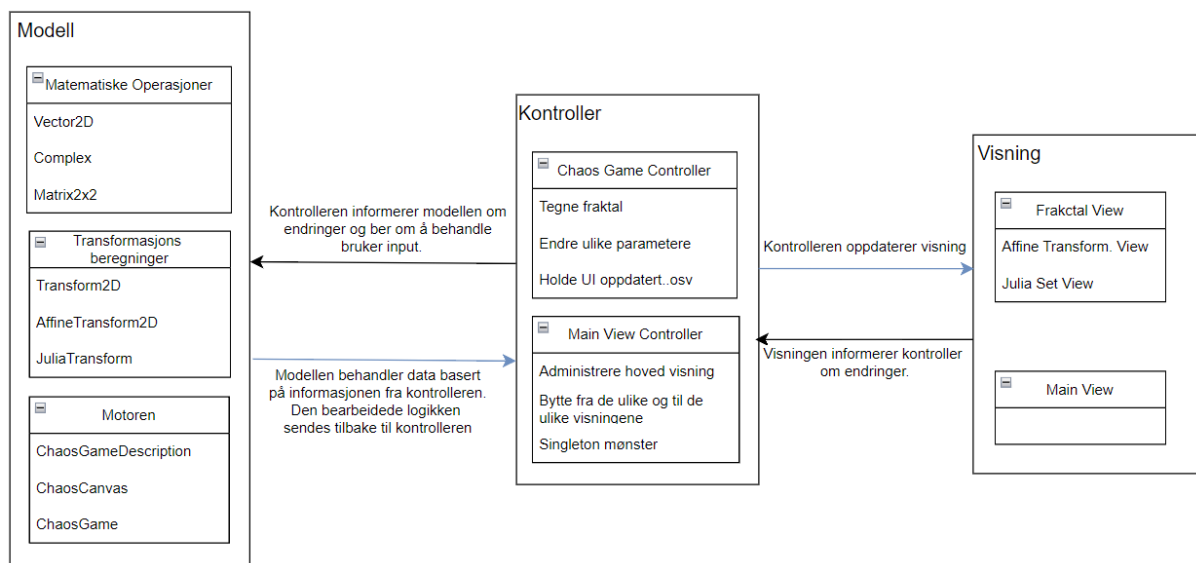
Denne forkortede versjonen bevarer de essensielle detaljene om utviklingsprosessen og organiseringen etter gruppeoppløsningen.

4 RESULTAT

I denne delen vil resultatene representeres ved hjelp av ulike diagrammer. Disse diagrammene er delt opp i ulike deler, for å gi et helhetlig bilde på systemet.

4.1 Teknisk Design

4.1.1 Systemarkitektur



Figur 2: Illustrasjon av Systemarkitekturen som har blitt benyttet i systemet.

Applikasjonen er bygget som en desktop-applikasjon ved bruk av MVC-arkitektur (se Teori, avsnitt "MVC Arkitektur"). Ved å følge dette arkitekturmønsteret sikres en klar separasjon av bekymringer.

Modell:

Modellen håndterer visningen av fraktaler og består av tre hovedroller: matematiske operasjoner, transformasjonsberegninger og motoren. Nøkkellasser inkluderer:

- **Matematiske operasjoner:**
 - **Vector2D:** Representerer en vektor eller et punkt i planet.
 - **Complex:** Representerer et komplekst tall.
 - **Matrix2x2:** Representerer en 2-dimensjonal-matrise.
- **Transformasjonsberegninger:**

- **Transform2D:** Et grensesnitt for matematiske transformasjoner (se Teori, avsnitt "Abstraksjon, Abstrakte Klasser og Grensesnitt").
- **AffineTransform2D:** Representerer en affine transformasjon.
- **JuliaTransform:** Representerer en kompleks transformasjon.
- **Motoren:**
 - **ChaosGameDescription:** Beskriver et kaospill med transformasjoner og parametere.
 - **ChaosCanvas:** Tegner fraktalen og oppdaterer grafikken.
 - **ChaosGame:** Håndterer logikken for å generere fraktaler ved å anvende affine transformasjoner på et punkt.

Visning:

Visningen presenterer data til brukeren og oppdateres basert på modellens tilstand (se Teori, avsnitt "MVC Arkitektur").

- **FractalView:** En abstrakt klasse som har subclassene **JuliaSetView** og **AffineTransformView**. Disse viser fraktaler og relevant informasjon, og er designet for enkel utvidelse (se Teori, avsnitt "Arv").
- **MainView:** Hoved visningen som lar brukeren navigere mellom ulike fraktaler.

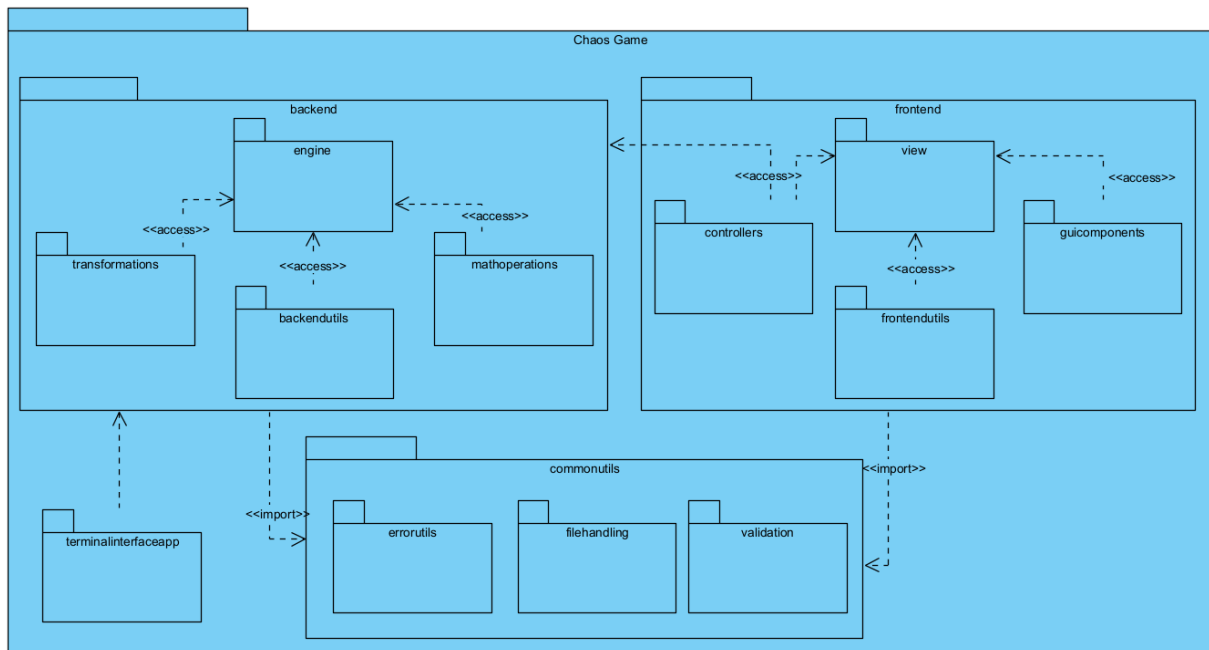
Kontroller:

Kontrolleren håndterer brukerinput og oppdaterer modellen og visningen (se Teori, avsnitt "MVC Arkitektur").

- **ChaosGameController:** Håndterer interaksjoner for fraktale operasjoner, inkludert tegning og parameterendringer.
- **MainViewController:** Administrerer hoved visningen og bruker Singleton-mønsteret for å opprettholde applikasjonstilstanden.

Denne strukturen sikrer en klar separasjon av bekymringer, gjør koden modulær, og letter vedlikehold og utvidelse av applikasjonen, i tråd med prinsippene for lav kobling og høy kohesjon (se Teori, avsnitt "Kobling og Kohesjon").

4.1.2 Pakkestruktur for Applikasjonen



Figur 3: Pakkediagram som illustrerer strukturen mellom pakkene

For å organisere koden på en ryddig måte, er applikasjonen delt inn i forskjellige pakker [Figur 3]. Hver med et spesifikt ansvar, noe som gjør det enkelt å finne og vedlikeholde koden. Kodens pakkestruktur er delt i hoved pakker:

- *frontend*: Inneholder alle pakker og klasser som håndterer visningsdelen av applikasjonen, inkludert kontrollerklasser som kommuniserer med modellen.
- *commonutil*: Felles pakke som inneholder hjelpeklasser som brukes av modellen, kontrolleren og visningen.
- *backend*: Består av pakker som håndterer applikasjonens forretningslogikk.

Denne pakkestrukturen ble valgt for å speile arkitekturmønsteret best mulig og sikre en modularitet og vedlikeholdbar kodebase.

Begrunnelse for Valgt Arkitektur

Pakke-arkitekturen ble valgt av flere grunner:

- *Separation of Concerns (SoC)*: Skiller forretningslogikk fra brukergrensesnitt og felles verktøy, noe som forbedrer organiseringen av koden.

- *Modularitet:* Gjør det enkelt å vedlikeholde og utvide applikasjonen ved å ha klare ansvarsområder for hver pakke.
- *Testbarhet:* Isolering av forretningslogikk i backend gjør det lettere å teste applikasjonen uavhengig av brukergrensesnittet.
- *Gjenbrukbarhet:* Felles verktøy i *commonutil* kan gjenbrukes av både backend og *frontend*, noe som reduserer duplisering av kode.

Ved å følge denne arkitekturen kan vi utvikle en robust og vedlikeholdbar applikasjon som oppfyller kravene i oppgaven og sikrer en god brukeropplevelse.

4.2 Implementasjon

4.2.1 Oversikt over Eksterne Biblioteker og Rammeverk

Applikasjonen benytter flere eksterne biblioteker og rammeverk. Prosjektet administreres ved hjelp av Maven, som håndterer avhengigheter og bygger applikasjonen.

Maven: Brukt som byggesystem og avhengighetshåndterer. Maven ble valgt på grunn av kravene av oppgavebeskrivelsen. Samtidig har den evnen til å forenkle byggeprosessen og håndtere prosjekter med mange avhengigheter.

JavaFX: Brukt for å lage det grafiske brukergrensesnittet. JavaFX ble valgt på grunn av funksjonaliteter og støtte for å lage gode og interaktive brukergrensesnitt.

- Brukte moduler:

- *javafx-controls* (versjon 21.0.2): Inneholder GUI-komponenter som knapper, tekstfelt og andre kontroller.
- *javafx-swing* (versjon 21.0.2): Gjør det mulig å innpasse Swing-komponenter i en JavaFX-applikasjon.

JUnit Jupiter: Brukt for enhetstesting av applikasjonen. JUnit ble valgt på grunn av dets enkle integrasjon med Java og brede støtte i utviklermiljøet.

- Brukte moduler:

- *junit-jupiter* (versjon 5.10.1): Den nyeste versjonen av JUnit som gir et kraftig rammeverk for å skrive og kjøre tester.

Gson: Brukt for serialisering og deserialisering av JSON-data. Gson ble valgt på grunn av sin enkelhet og effektivitet i å konvertere Java-objekter til JSON og vice versa.

- Brukte moduler:

- *gson* (versjon 2.9.1): Hovedbiblioteket for JSON-håndtering.

Mockito: Brukt for å lage mock-objekter for enhetstesting. Mockito ble valgt fordi det er et kraftig verktøy for å isolere enheter i testing og for å simulere avhengigheter.

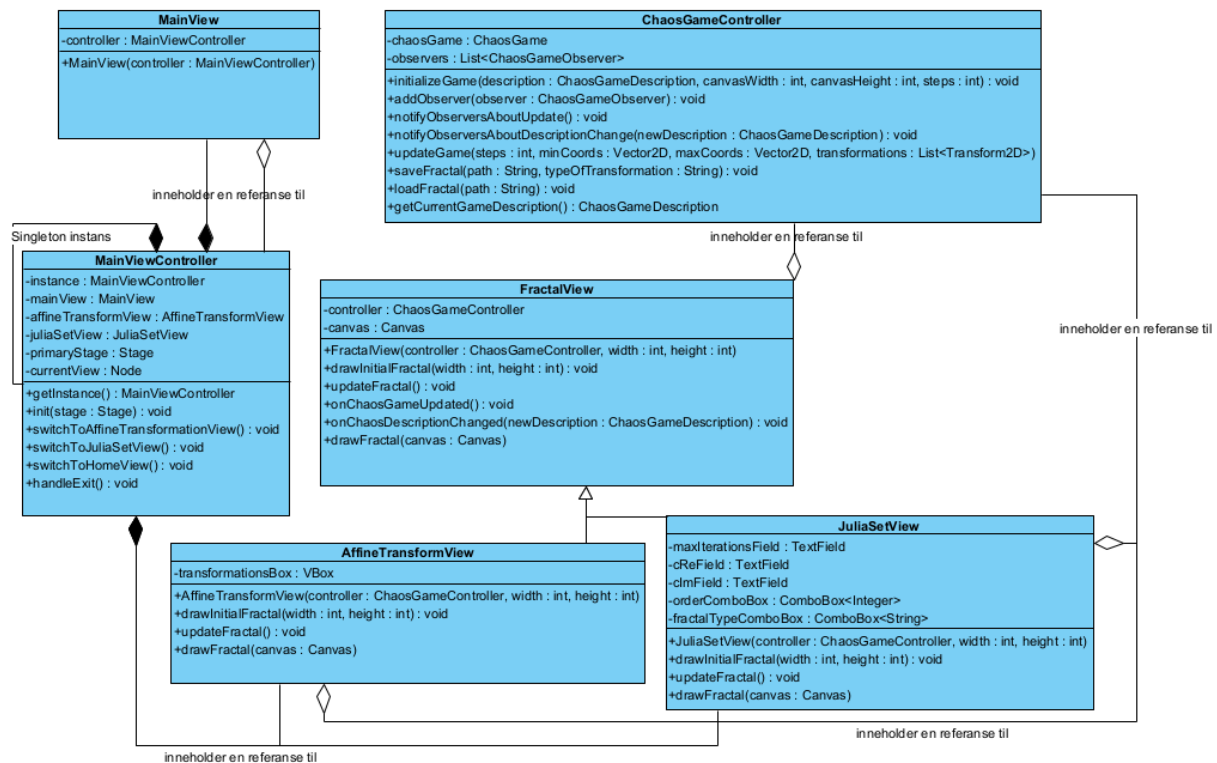
- Brukte moduler:

- *mockito-core* (versjon 5.5.0): Hovedbiblioteket for mocking.
- *mockito-junit-jupiter* (versjon 5.5.0): Integrasjon mellom Mockito og JUnit Jupiter for å gjøre testskriving enklere.

Disse bibliotekene og rammeverkene bidro til en mer effektiv utviklingsprosess.

4.2.2 Klassediagrammer

4.2.2.1 Brukerinteraksjon (View og Controller)



Figur 4: Klassediagram for Brukerinteraksjon

Klassediagrammet [Figur 4] representerer strukturen og relasjonene mellom ulike klasser som håndterer fraktal beregning og brukerinteraksjon i systemet. Diagrammet viser hvordan brukeren interagerer med systemet gjennom visninger og kontroller, og hvordan disse komponentene samarbeider for å beregne og vise fraktaler.

MainView er hoved visningen som inneholder en referanse til **MainViewController**. **MainView** fungerer som inngangspunktet for brukerinteraksjon og viser ulike visninger basert på brukerens valg. **MainViewController** er en singleton-klasse som administrerer hoved logikken og navigasjonen i applikasjonen, og tilbyr metoder for å initialere hovedvinduet, bytte visninger og håndtere applikasjonsavslutning.

FractalView er en abstrakt klasse som gir felles funksjonalitet for alle fraktal visninger. Den inneholder referanser til **ChaosGameController** og **Canvas**, og tilbyr metoder for å oppdatere og tegne fraktaler. **AffineTransformView** og **JuliaSetView** arver fra **FractalView** og

representerer spesifikke fraktal visninger, og er subbklasser av FractalView.

AffineTransformView håndterer affine transformasjoner, mens JuliaSetView håndterer Julia-sett, hver med sine unike felter og metoder for tegning og oppdatering.

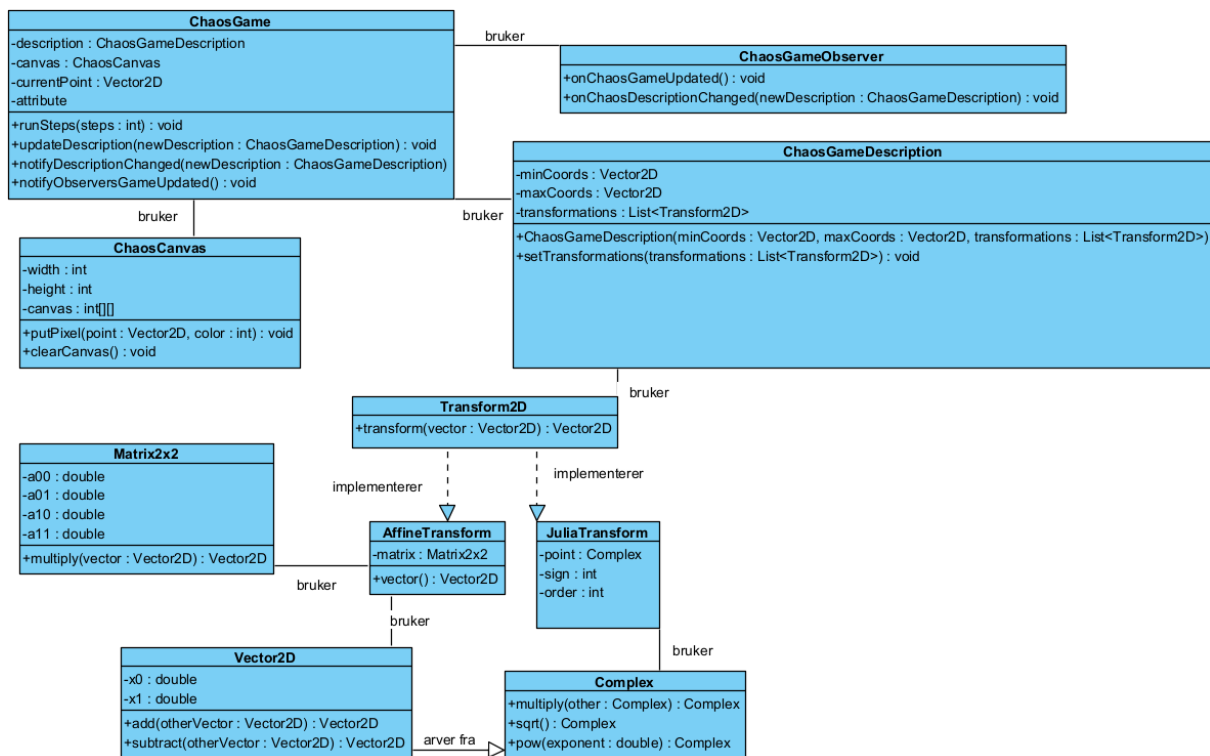
ChaosGameController håndterer logikken for fraktal beregning. Den har referanser til ChaosGame og en liste over ChaosGameObserver, og tilbyr metoder for å initialere spillet, oppdatere spillet og varsle observere.

ChaosGame representerer spillet som beregner fraktaler, håndterer opprettelse av canvas, transformasjoner av punkter og tegning av fraktaler.

ChaosCanvas er ansvarlig for å tegne fraktaler på et 2D-array som representerer canvas.

Samlet viser klassediagrammet hvordan brukerinteraksjon håndteres gjennom et system av visninger og kontroller som samarbeider for å beregne og vise fraktaler. Denne strukturen er i tråd med prinsippene om lav kobling, høy kohesjon, arv og polymorfisme (se Teori, avsnitt "Kobling og Kohesjon", "Arv" og "MVC Arkitektur"). Ved å bruke MVC-arkitekturen sikres en klar separasjon av bekymringer, noe som gjør koden mer modulær og lettere å vedlikeholde (se Teori, avsnitt "MVC Arkitektur").

4.2.2.2 Forretningslogikk (Modell)



Figur 5: Klassediagram for Forretningslogikken (Modellen)

Klassediagrammet [Figur 5] viser strukturen og relasjonene mellom klassene som utgjør forretningslogikken i systemet, med fokus på kaospillet for å generere fraktaler. Diagrammet illustrerer hvordan klasser som representerer transformasjoner, vektorer og komplekse tall brukes til å beregne og tegne fraktaler.

ChaosGame er kjernen som styrer fraktal beregningene, og inneholder referanser til ChaosGameDescription, som beskriver spillet, og ChaosCanvas, som er tegneflaten.

Viktige metoder inkluderer

- *runSteps* for å utføre beregningstrinn,
- *updateDescription* for å oppdatere spill beskrivelsen
- *notifyObserversGameUpdated* for å varsle observere om oppdateringer.

ChaosCanvas tegner fraktalen ved å bruke dimensjoner og et 2D-array for tegneområdet.

Metoder som *putPixel* tegner piksler, mens *clearCanvas* tømmer tegneflaten.

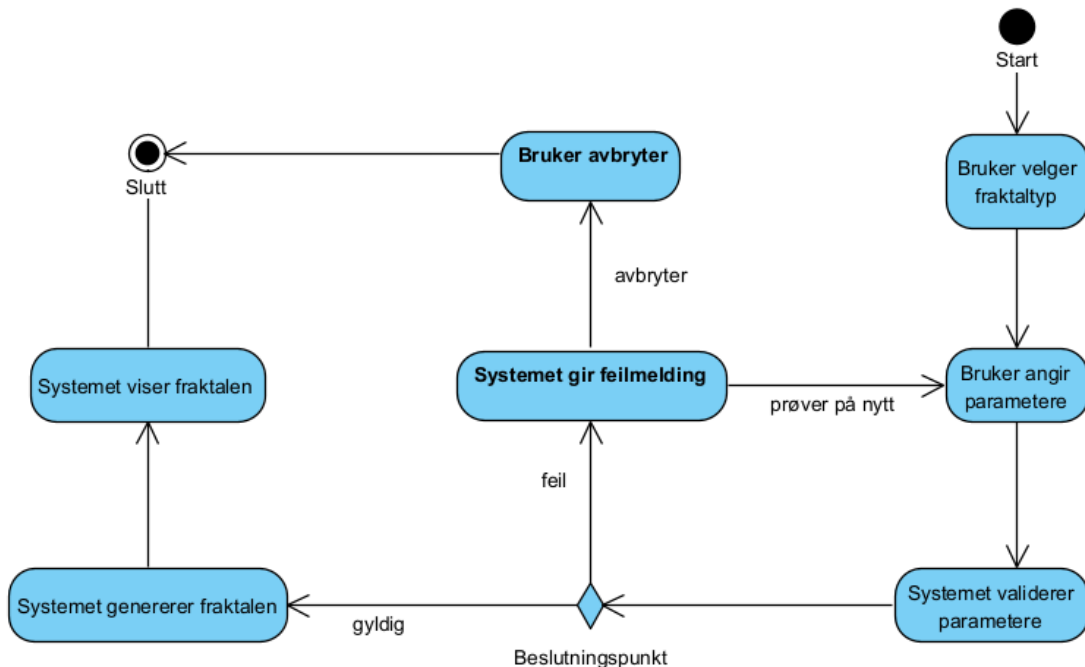
ChaosGameDescription spesifiserer spilllets minimums- og maksimumskoordinater og en liste over transformasjoner. Metoden *setTransformations* setter transformasjonene.

ChaosGameObserver er et grensesnitt med metoder for å håndtere spilloppdateringer, mens Transform2D er et grensesnitt for 2D-transformasjoner med metoden *transform*. AffineTransform og JuliaTransform implementerer Transform2D. AffineTransform bruker en 2x2 matrise og en translasjonsvektor for affine transformasjoner, mens JuliaTransform bruker komplekse tall for Julia-sett-transformasjoner. Matrix2x2 representerer en 2x2 matrise, og Vector2D representerer en 2D-vektor. Complex representerer komplekse tall med metoder for komplekse operasjoner.

Dette klassesdiagrammet viser hvordan komponentene i modellen samarbeider for å beregne og tegne fraktaler. ChaosGame bruker transformasjoner definert i ChaosGameDescription, mens ChaosCanvas tegner fraktalen basert på disse beregningene. Matematiske verktøy som Vector2D og Complex muliggjør disse beregningene, og transformasjoner utføres av AffineTransform og JuliaTransform. Denne strukturen sikrer at systemet kan generere og vise komplekse fraktaler basert på brukerens spesifikasjoner.

Referanser til teori inkluderer prinsippene om lav kobling og høy kohesjon, som sikrer at klassene er godt organiserte og enkle å vedlikeholde (se Teori, avsnitt "Kobling og Kohesjon"), og bruk av arv og polymorfisme for fleksibel og gjenbrukbar kode (se Teori, avsnitt "Arv"). MVC-arkitekturen fremheves for å separere forretningslogikk fra brukergrensesnitt (se Teori, avsnitt "MVC Arkitektur").

4.2.3 Aktivitetsdiagram



Figur 6: Aktivitetsdiagram for Opprettelse av Fraktaler

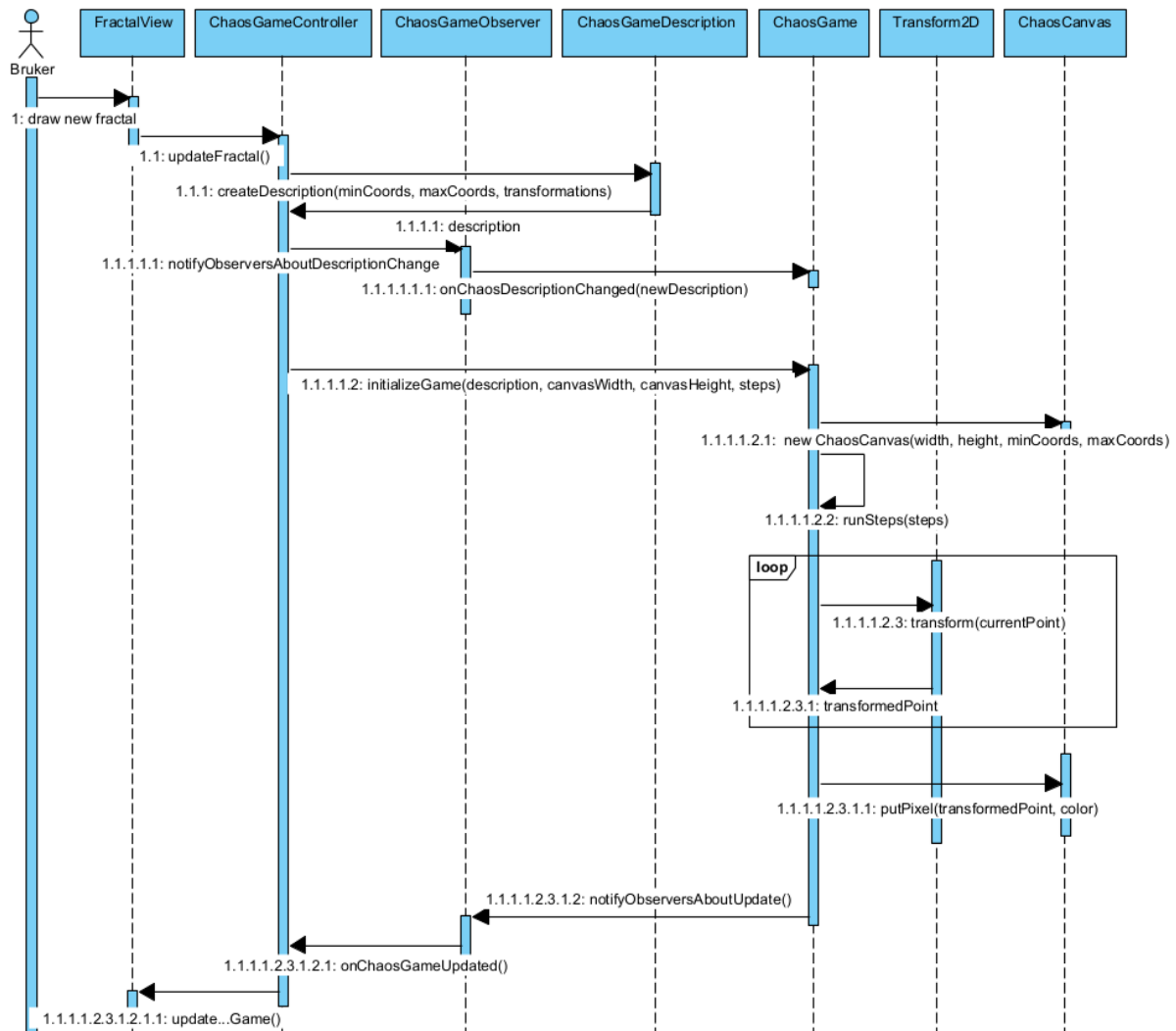
Aktivitetsdiagrammet [Figur 6] viser trinnene i prosessen for å opprette en fraktal i systemet, fra starten når brukeren velger fraktaltyp, til sluttresultatet der fraktalen vises på skjermen. Prosessen starter med at brukeren velger fraktaltyp fra tilgjengelige alternativer. Deretter angir brukeren nødvendige parametere, som kan inkludere koordinater, antall iterasjoner, og andre relevante innstillinger. Systemet validerer deretter de angitte parameterne for å sikre at de er gyldige og innenfor akseptable områder.

Hvis parameterne er gyldige, genererer systemet fraktalen basert på de angitte innstillingene. Når fraktalen er generert, vises den på skjermen for brukeren. Prosessen avsluttes her med at brukeren kan se og utforske den genererte fraktalen.

Hvis systemet finner feil i de angitte parameterne, gir det en feilmelding til brukeren. Brukeren får da muligheten til å rette opp parameterne og prøve på nytt. Alternativt kan brukeren velge å avbryte prosessen, som også avslutter aktiviteten.

Denne flyten sikrer at systemet støtter brukeren gjennom hele prosessen, fra valg av fraktaltyp til visning av resultatet, med nødvendige valideringer og tilbakemeldinger underveis.

4.2.4 Sekvensdiagram



Figur 7: Sekvensdiagram, Generering av Fraktaler

Diagrammet i [Figur 7] viser sekvensen av meldinger mellom brukergrensesnittet, kontrolleren og forretningslogikken for å generere og vise fraktaler. Diagrammet er forenklet til de viktigste delene.

Når brukeren klikker på "draw new fractal" i brukergrensesnittet, starter FractalView prosessen ved å kalle *updateFractal*, som henter nødvendige parametere og kaller ChaosGameController for å oppdatere spillet. ChaosGameController oppretter en ny ChaosGameDescription og varsler observere om endringen.

FractalView, som implementerer ChaosGameObserver, mottar meldingen *onChaosDescriptionChanged* og oppdaterer sin tilstand. Deretter kaller

ChaosGameController *initializeGame* på ChaosGame, som initialiserer spillet og setter opp en ny ChaosCanvas.

ChaosGame kjører *runSteps* for å beregne fraktalens punkter og plasserer dem på canvasen med *putPixel*. Etter at stegene er fullført, varsler ChaosGame observere med *notifyObserversAboutUpdate*.

FractalView mottar meldingen *onChaosGameUpdated* og oppdaterer visningen ved å tegne de nye punktene. Til slutt kaller FractalView *updateGame* på ChaosGameController for å sikre at alle endringer er reflektert.

Denne sekvensen viser hvordan brukerinteraksjon, kontroller og forretningslogikk samarbeider for å generere og vise fraktaler, i tråd med prinsippene om lav kobling og høy kohesjon (se Teori, avsnitt "Kobling og Kohesjon") og MVC-arkitekturen (se Teori, avsnitt "MVC Arkitektur").

4.3 Testing

4.3.1 Enhetstesting (Unit Testing)

For enhetstesting brukte jeg JUnit Jupiter og Mockito for å teste kritiske komponenter og forretningslogikk i applikasjonen, både positive og negative scenarier. Testene inkluderte alle klasser i modellen (backend) og hjelpeklasser (commonutils).

4.3.2 Ytelsestesting

Ytelsestesting ble utført for å evaluere applikasjonens belastningstoleranse. Jeg testet hvor mange iterasjoner for tegning av fraktaler som kunne utføres før ytelsen avtok, for å identifisere flaskehalser og optimalisere koden.

4.3.3 Regresjonstesting

Regresjonstesting sikrer at nye funksjoner eller endringer ikke introduserer feil i eksisterende funksjoner. Jeg skrev nye enhetstester eller tilpasset eksisterende tester for nye funksjoner og utførte manuell testing av brukergrensesnittet.

4.3.4 Manuell Testing

Manuell testing ble utført for å identifisere feil som automatiserte tester ikke fanget opp. Etter hver kodeendring gjennomførte jeg manuell testing av brukerinteraksjoner og funksjonalitet for å sikre at ingen nye feil ble introdusert.

4.3.5 Brukertest

Brukertest ble utført for å evaluere brukervennligheten. Jeg fikk tilbakemeldinger fra bekjente, inkludert familie og venner, på brukergrensesnittet for å identifisere feil eller forbedringsområder.

5 DRØFTING

5.1 Drøfting av løsning/design

Prosjektet utviklet en desktop-applikasjon for å generere fraktaler ved bruk av kaosteori, implementert med MVC-arkitektur for å skille forretningslogikk fra brukergrensesnittet. Alle funksjonelle krav ble oppfylt, inkludert generering av ulike fraktaler, dynamisk parameterjustering og lagring av fraktalkonfigurasjoner. Ekstra funksjoner som fargelegging av fraktaler, ulike transformasjonssannsynligheter, høyere ordens transformasjoner og Mandelbrot-mengde ble også implementert. Data persistering med JSON, animasjon av fraktaler, zooming og brukerhjelp ble ikke fullført på grunn av tidsbegrensninger.

En stor utfordring var å sikre applikasjonens responsivitet under høy belastning, noe som krevde optimalisering av algoritmer. En annen utfordring var å sikre lav kobling og høy kohesjon, oppnådd gjennom nøye planlegging av klassearkitekturen (se Teori, avsnitt "Kobling og Kohesjon").

Designprinsipper

Lav Kobling og Høy Kohesjon: Ved å gjøre felter private og tilgjengelige via getters, og bruke grensesnitt for klassekommunikasjon, ble lav kobling oppnådd. ChaosGameDescription er for eksempel uavhengig av spesifikke transformasjoner ved å bruke et generisk grensesnitt. Høy kohesjon ble sikret ved å designe klasser med veldefinerte ansvarsområder, som Vector2D som kun håndterer vektoroperasjoner.

Arv og Polymorfisme: Brukt for å fremme gjenbruk av kode og forbedre organiseringen av klasser. FractalView fungerer som en felles baseklasse for ulike fraktalvisninger som JuliaSetView og AffineTransformView, som deler felles funksjonalitet og spesialiseres for spesifikke behov.

Abstraksjon, Abstrakte Klasser og Grensesnitt: Brukt for å forenkle systemer og skjule irrelevante detaljer. Abstrakte klasser som FractalView og grensesnitt som Transform2D definerer felles atferd for subklasser.

MVC Arkitektur: Skiller modell, visning og kontroller, og forbedrer organiseringen og vedlike holdbarheten av koden (se Teori, avsnitt "MVC Arkitektur").

Fremtidige Forbedringer

Fremtidig utvikling kan inkludere ytterligere ytelsesoptimalisering, forbedring av brukergrensesnittet med interaktive guider og avanserte visualiseringer, og vurdering av andre arkitekturmodeller som MVVM for bedre separasjon av bekymringer.

5.2 Drøfting av prosess

Etter at gruppemedlemmet mitt sluttet, jobbet jeg alene, noe som skapte utfordringer, særlig med tid og arbeidsmengde. I starten hadde vi en strukturert prosess med hyppige commits og kodegjennomganger, og jeg opprettholdt denne strukturen så godt som mulig. Jeg sørget for å ha en fungerende applikasjon klar til hver godkjenningssamtale.

Hva fungerte bra?

1. **Iterativ utvikling:** Regelmessige milepæler og tilbakemeldinger hjalp med å løse problemer tidlig.
2. **Dokumentasjon:** Nøyaktig dokumentasjon holdt oversikt over fremdriften.
3. **Testing:** Enhetstester og manuelle tester opprettholdt kvaliteten på koden.

1. Hva fungerte mindre bra?

2. **Tidsstyring:** Balansen mellom utviklingsarbeid og andre prosjekter var utfordrende.
3. **Arbeidsmengde:** Å håndtere alle oppgaver alene var overveldende.

Hva ville jeg gjort annerledes?

1. **Detaljerte milepæler:** Små og realistiske milepæler ville ha hjulpet med jevn fremdrift.
2. **Bedre tidsstyring:** En detaljert tidsplan med buffertid for uforutsette hendelser ville ha hjulpet.
3. **Automatisering:** Investering i automatiserte tester og oppgaver ville ha spart tid og sikret høyere kvalitet.

6 KONKLUSJON

Til tross for utfordringer fulgte jeg hovedplanen og metodikken. Den iterative prosessen fungerte godt, men tidsstyring og arbeidsmengde var utfordrende. Med bedre planlegging kunne resultatene vært enda bedre. Refleksjonen gir innsikt i forbedringsområder for fremtidige prosjekter, noe som vil forbedre både effektiviteten og kvaliteten på arbeidet.

6.1 *Drøfting av bruken av KI-verktøy*

Jeg brukte ChatGPT som læringsmiddel i dette prosjektet. Verktøyet var nyttig for å få raske svar på tekniske spørsmål, få forslag til kodeforbedringer og forstå komplekse konsepter. En svakhet var at noen svar kunne være generelle og trengte videre tilpasning til spesifikke problemer. Ellers bidro ChatGPT til effektiv problemløsning og læring underveis i utviklingsprosessen.

REFERANSER

- [1] D. J. Barnes and M. Kölling, "Objects First with Java: A Practical Introduction using BlueJ," 6th ed. Pearson, 2017.
- [2] GeeksforGeeks (07 Nov, 2023). *Coupling and Cohesion – Software Engineering* [Online]: <https://www.geeksforgeeks.org/software-engineering-coupling-and-cohesion/>
- [3] GeekforGeeks (07 May, 2024). *Differences between Coupling and Cohesion-Software Engineering* [Online]: <https://www.geeksforgeeks.org/software-engineering-differences-between-coupling-and-cohesion/>
- [4] Baeldung, Ganesh Pagade (09 Nov, 2022) *Difference Between Cohesion and Coupling* [Online]: <https://www.baeldung.com/cs/cohesion-vs-coupling>
- [5] GeekforGeeks (14 May, 2024). *Inheritance in Java* [Online]: <https://www.geeksforgeeks.org/inheritance-in-java/>
- [6] upGrad, Pavan Vadapalli (11 March, 2024). *Inheritance in Java* [Online]: <https://www.upgrad.com/tutorials/software-engineering/java-tutorial/inheritance-in-java/>
- [7] Javatpoint. *MVC Architecture in Java* [Online]: <https://www.javatpoint.com/mvc-architecture-in-java>